

Saga Orchestration Flow (Happy Path)

Architecture

```
Client
  |
  v
Order Service
  |
  | OrderCreatedEvent
  v
Saga Service
  |
  | ReserveInventoryCommand
  v
Inventory Service
  |
  | InventoryReservedEvent
  v
Saga Service
  |
  | ProcessPaymentCommand
  v
Payment Service
  |
  | PaymentCompletedEvent
  v
Saga Service
  |
  | Saga COMPLETED
```

Step 1 : Client Creates Order

The client sends a request to create an order.

```
POST /orders
```

Example Request

```
{
  "orderId": "ORD1002",
  "product": "Laptop",
  "quantity": 1
}
```

Step 2 : Order Service Saves Order

The Order Service creates an Order entity.

```
Order order = Order.builder()
    .orderId(event.getOrderId())
    .product(event.getProduct())
    .quantity(event.getQuantity())
    .status("CREATED")
    .build();

repository.save(order);
```

Database

```
orders
-----
order_id    product    quantity    status
-----
ORD1002     Laptop     1           CREATED
```

Step 3 : Create Outbox Event

Within the same transaction, an Outbox record is created.

```
OutboxEvent outbox = OutboxEvent.builder()
    .eventType("ORDER_CREATED")
    .payload(mapper.writeValueAsString(event))
    .published(false)
    .createdAt(LocalDateTime.now())
    .build();
```

```
outboxRepository.save(outbox);
```

Database

```
outbox
-----
id   eventType      published
-----
1    ORDER_CREATED  false
```

The Outbox Pattern guarantees that if the database transaction commits successfully, the event will eventually be published to Kafka.

Step 4 : Outbox Publisher Reads Pending Events

A scheduler periodically checks for unpublished events.

```
@Scheduled(fixedRate = 5000)
public void publishEvents() {

    List<OutboxEvent> events =
        outboxRepository.findByPublishedFalse();

}
```

Step 5 : Publish OrderCreatedEvent

The Outbox event is converted back into an OrderCreatedEvent and sent to Kafka.

```
kafkaTemplate.send(
    "order-created",
    event.getOrderId(),
    event
);
```

Topic

```
order-created
```

Step 6 : Mark Outbox as Published

After successful publishing,

```
event.setPublished(true);  
outboxRepository.save(event);
```

Database

```
published = true
```

Step 7 : Saga Receives OrderCreatedEvent

Saga Orchestrator listens to the event.

```
@KafkaListener(  
    topics = "order-created",  
    groupId = "saga-group"  
)  
public void consume(OrderCreatedEvent event)
```

Step 8 : Saga Starts

A new Saga instance is created.

```
repository.save(  
    new SagaState(  
        sagaId,  
        event.getOrderId(),  
        "STARTED"    )  
)
```

```
)  
);
```

Database

```
saga_state  
-----  
saga_id      order_id      status  
-----  
UUID         ORD1002      STARTED
```

Step 9 : Saga Sends ReserveInventoryCommand

Saga instructs Inventory Service to reserve stock.

```
producer.sendReserveInventory(  
  
    new ReserveInventoryCommand(  
  
        sagaId,  
        orderId,  
        product,  
        quantity  
    )  
);
```

Kafka Topic

```
reserve-inventory
```

Step 10 : Inventory Service Receives Command

```
@KafkaListener(  
    topics = "reserve-inventory",  
    groupId = "inventory-group"  
)
```

Inventory receives the ReserveInventoryCommand.

Step 11 : Idempotency Check

Duplicate messages are ignored.

```
processedEventRepository.existsById(  
    command.getSagaId()  
);
```

This prevents processing the same Kafka message multiple times.

Step 12 : Reduce Inventory

```
inventory.setStock(  
  
    inventory.getStock()  
    - command.getQuantity()  
  
);  
  
repository.save(inventory);
```

Database

```
inventory  
-----  
Laptop  
Stock = Stock - 1
```

Step 13 : Publish InventoryReservedEvent

Inventory Service notifies Saga.

```
InventoryReservedEvent event =
```

```
        new InventoryReservedEvent(  
            sagaId,  
            orderId,  
            product,  
            quantity  
        );  
  
kafkaTemplate.send(  
    "inventory-reserved",  
    orderId,  
    event  
);
```

Topic

```
inventory-reserved
```

Step 14 : Saga Receives InventoryReservedEvent

```
@KafkaListener(  
    topics = "inventory-reserved",  
    groupId = "saga-group"  
)
```

Saga updates its state.

```
state.setStatus(  
    "INVENTORY_RESERVED"  
);  
  
repository.save(state);
```

Database

```
status = INVENTORY_RESERVED
```

Step 15 : Saga Sends ProcessPaymentCommand

```
producer.sendProcessPayment(  
  
    new ProcessPaymentCommand(  
  
        sagaId,  
        orderId,  
        1000.0  
  
    )  
);
```

Kafka Topic

```
process-payment
```

Step 16 : Payment Service Receives Command

```
@KafkaListener(  
    topics = "process-payment",  
    groupId = "payment-group"  
)
```

Payment Service starts processing payment.

Step 17 : Save Payment

```
Payment payment =  
  
    Payment.builder()  
  
        .orderId(command.getOrderId())  
        .amount(command.getAmount())  
        .status("COMPLETED")  
  
        .build();
```



```
repository.save(payment);
```

Database

```
payment
-----
order_id    amount    status
-----
ORD1002     1000     COMPLETED
```

Step 18 : Publish PaymentCompletedEvent

```
PaymentCompletedEvent event =

    new PaymentCompletedEvent(

        command.getSagaId(),
        command.getOrderId(),
        command.getAmount()

    );

kafkaTemplate.send(

    "payment-completed",
    command.getOrderId(),
    event

);
```

Topic

```
payment-completed
```

Step 19 : Saga Receives PaymentCompletedEvent

```
@KafkaListener(
    topics = "payment-completed",
```

```
        groupId = "saga-group"  
    )
```

Saga updates its state.

```
state.setStatus("COMPLETED");  
repository.save(state);
```

Database

```
status = COMPLETED
```

Step 20 : Saga Successfully Completes

The distributed transaction is complete.

Final Database State

Orders

```
ORD1002  
Status = CREATED
```

Inventory

```
Laptop Stock = Reduced
```

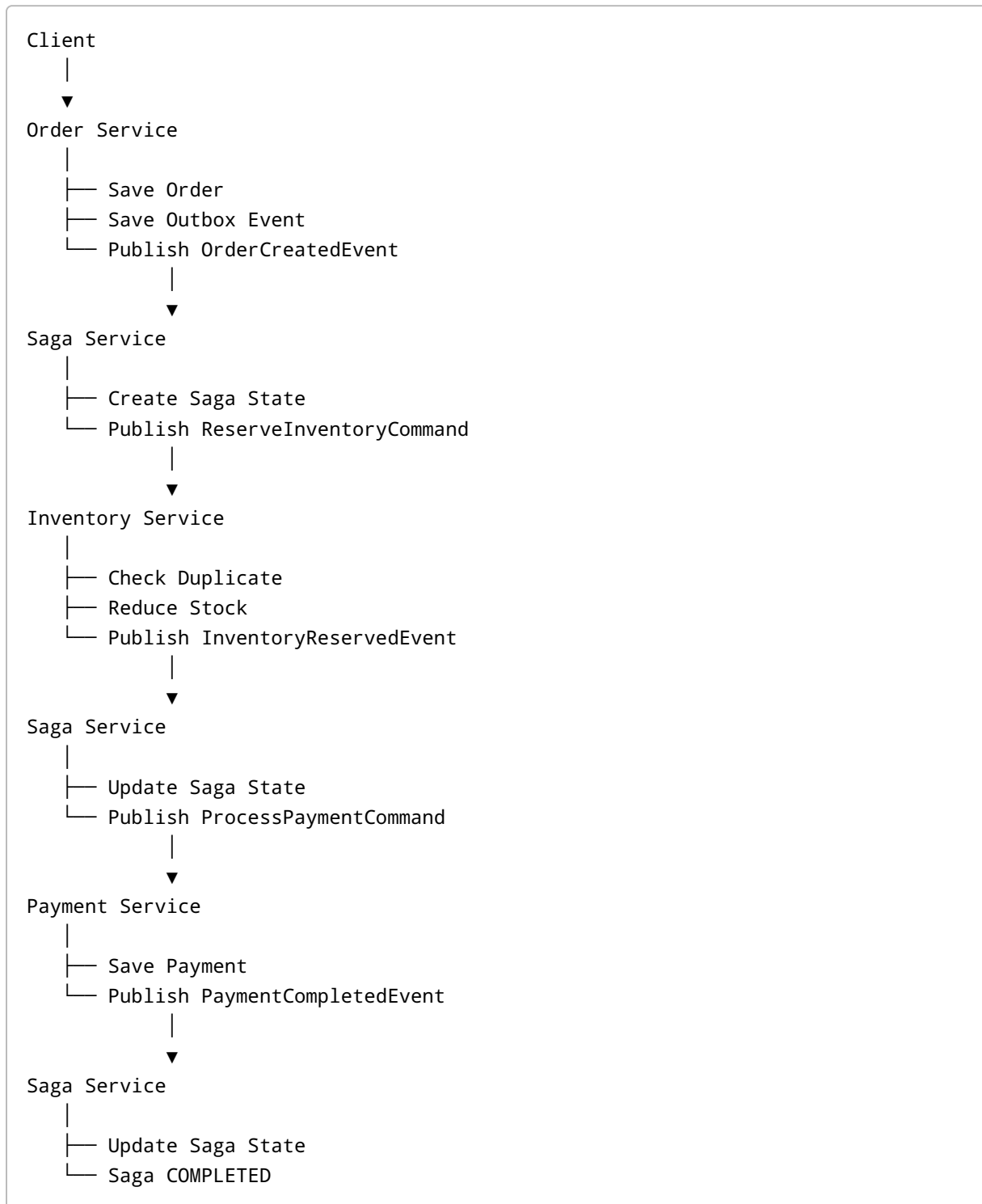
Payment

```
Payment Status = COMPLETED
```

Saga

```
Saga Status = COMPLETED
```

Final Event Flow



Design Patterns Used

- Transactional Outbox Pattern
 - Saga Orchestration Pattern
 - Idempotent Consumer Pattern
 - Event-Driven Architecture
 - Kafka Messaging
 - Database per Service Pattern
 - Eventual Consistency
-

Current Project Progress

- ✓ Order Service
- ✓ Outbox Pattern
- ✓ Kafka Producer
- ✓ Kafka Consumer
- ✓ Saga Orchestrator
- ✓ Inventory Service
- ✓ Payment Service
- ✓ Idempotent Consumer
- ✓ Saga State Management
- ✓ Happy Path Saga Completed

Next Phase

- Payment Failed Event
- ReleaseInventoryCommand
- InventoryReleasedEvent
- CancelOrderCommand
- OrderCancelledEvent
- Full Compensation Saga (Rollback Flow)